

CHAPTER 3. SYSTEM IMPLEMENTATION

3.1 Implementation Overview

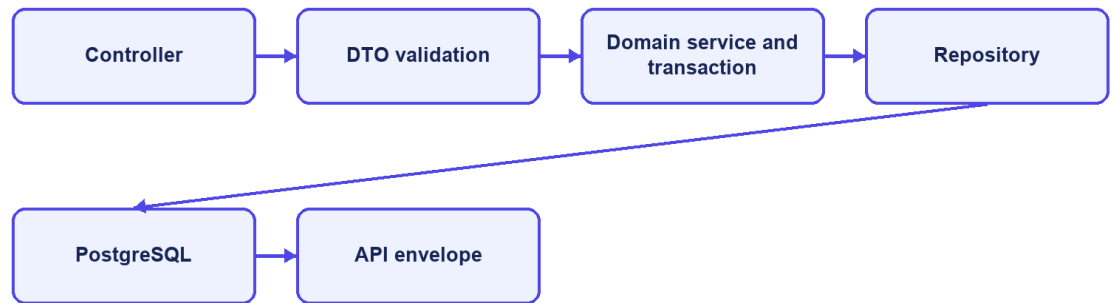
CareerFit is implemented as two application projects and a shared runtime environment. The backend is a Java 21 application based on Spring Boot 3.2.5. The frontend is a React 18 single-page application written in TypeScript and built with Vite. PostgreSQL is the transactional database, Flyway owns schema migration, and Docker Compose provides the reproducible local database and optional backend container. The implementation follows the modular-monolith design established in Chapter 2: domain packages share one backend process while retaining controllers, services, repositories, entities, and DTOs for each functional area.

Table 3.1. Main implementation technologies

Layer	Technology	Implementation purpose
Backend language/runtime	Java 21	Domain logic, API, background processing, and integrations
Application framework	Spring Boot 3.2.5	Dependency injection, web MVC, configuration, scheduling, and application lifecycle
Security	Spring Security and JWT 0.12.5	Stateless bearer authentication, role authorization, and JWT handling
Persistence	Spring Data JPA, PostgreSQL 16, Flyway	Entity persistence, queries, constraints, indexes, and versioned migrations
Document processing	PDFBox 3.0.2, Apache POI, Tesseract CLI	PDF/DOCX extraction and OCR fallback for images or scanned PDFs
API documentation	springdoc-openapi 2.5.0	OpenAPI documents and Swagger UI
Monitoring	Spring Actuator, Micrometer Prometheus registry	Health and HTTP/application metrics
Frontend	React 18.3, TypeScript 5.9, React Router 7, React Query 5	Role-based pages, typed API integration, navigation, and server-state queries
Visualization and UI	Recharts and Lucide React	Charts and interface icons
Build and test	Maven, npm/Vite, JUnit, Testcontainers, Playwright	Compilation, automated backend tests, integration environment, and browser E2E tests

The backend package root is `com.careerfit.backend`. Domain packages include `auth`, `candidate`, `cv`, `job`, `employer`, `matching`, `recommendation`, `application`, `feedback`, `automation`, `notification`, `analytics`, `admin`, `audit`, and `settings`. Shared response models, exceptions, text utilities, validation, security filters, and configuration are placed in common or configuration packages. This structure makes the dependency direction visible without adding network boundaries between modules.

Backend request path



CareerFit IT AutoPilot — thesis diagram

Figure 3.1. Backend module structure and request flow

3.2 Authentication and Authorization Implementation

3.2.1 Login and JWT Processing

AuthController exposes registration, password login, passwordless request/verification, and current-account operations. AuthService validates account state and credentials and delegates token creation to JwtService. Passwords are stored with BCryptPasswordEncoder. A successful login returns an access token and a user DTO containing the ID, email, full name, role, and verification state.

Each authenticated request passes through JwtAuthenticationFilter, a OncePerRequestFilter. The filter requires the Bearer prefix when an Authorization header is present, validates token structure and signature, extracts subject and role, rejects roles outside Candidate, Recruiter, and Administrator, and reloads the account from the database. Reloading prevents a cryptographically valid token from continuing to authorize a deleted or suspended account. The filter then creates a Spring Security authentication object with the corresponding ROLE_* authority.

UserIdResolutionFilter runs after JWT authentication. It resolves the persisted account ID from the authenticated email and stores the UUID as the userId request attribute. Controllers can therefore pass a stable identifier to services without accepting a user ID supplied by the browser. The implementation currently performs a user lookup in both filters; this is straightforward but creates duplicate database reads that could be consolidated later.

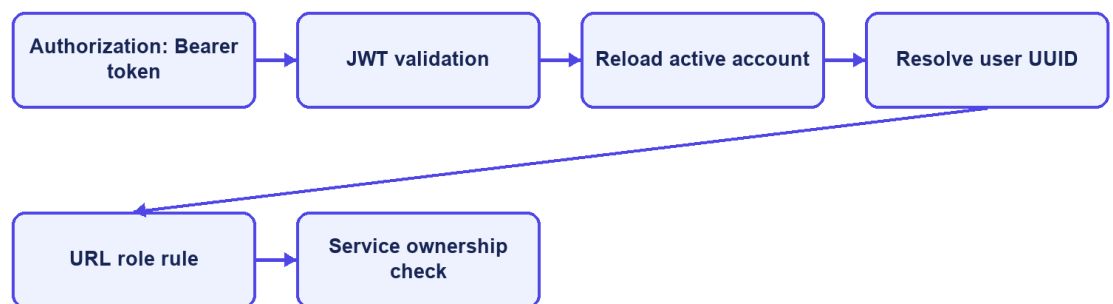
3.2.2 URL Rules and Ownership Checks

SecurityConfig uses a stateless session policy. Public access is limited to selected authentication operations, public job/employer/analytics GET routes, similar-job retrieval, health/metrics/OpenAPI routes, and email-action redemption. Candidate routes cover CV, matching, candidate profile, and applications. Recruiter routes cover owned jobs, candidate discovery, application review, employer profile, and export. Administrative routes require the Administrator role. Automation routes require authentication, with policy ownership resolved in services.

Role checks are necessary but insufficient for resources addressed by UUID. ApplicationService, for example, verifies that a requested CV belongs to the authenticated Candidate and that a Recruiter owns the Job before listing applicants, changing status, or inviting a candidate. The same pattern is required across identifier-based services. Errors are returned through shared security and application error writers so that 401 unauthenticated and 403 forbidden responses remain distinguishable.

The configuration disables CSRF because bearer tokens are used instead of a server-side browser session. CORS origins are loaded from application configuration. Frame denial, Content Security Policy, and HSTS headers are configured. HSTS is effective only when the application is served through HTTPS, so the local HTTP runtime does not itself demonstrate transport security.

JWT request processing



CareerFit IT AutoPilot — thesis diagram

Figure 3.2. JWT authentication and authorization boundaries

3.3 CV Ingestion and Validation

3.3.1 Upload and Manual Creation

CvController provides multipart upload, manual creation, CV listing, details, processing status, default selection, and deletion. CvIngestionService orchestrates the processing pipeline. Uploaded files are validated, stored through StorageService, and represented by a CV entity whose status makes background progress observable. Manual CV data is converted into labeled raw-text sections such as title, seniority, experience, skills, education, projects, certifications, and languages, after which it uses the same normalization and vectorization path.

Table 3.2. CV processing states

State	Meaning	Typical transition
UPLOADED	Metadata and source have been accepted	Upload/manual creation → validation
VALIDATING	File/content and quality checks are executing	Valid input → processing; hard error → failed
PROCESSING	Extraction, normalization, and vector creation are executing	Vector saved → scoring done
SCORING_DONE	CV vector is available and matching can be queried	Matching runs asynchronously against active jobs
FAILED	Processing cannot continue	Failure reason is persisted and audited

3.3.2 File Validation and Extraction

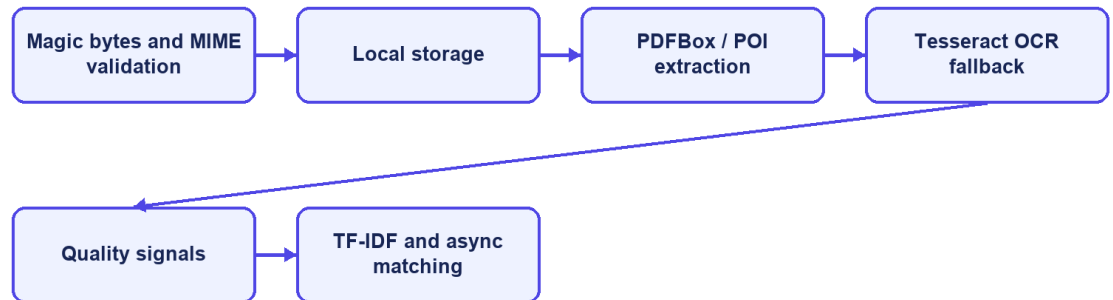
PdfExtractionService accepts PDF, PNG, JPG/JPEG, and DOCX. It checks the extension, MIME type, and leading magic bytes rather than trusting the filename alone. Empty input and mismatched content are rejected. PDFBox first attempts embedded-text extraction. Apache POI extracts DOCX text. Images use OCR directly. For a PDF with fewer than 50 extracted characters, the service renders up to the configured maximum number of pages and invokes a configurable Tesseract command. The default OCR language set is vie+eng, the default maximum is eight pages, and each OCR process has a timeout.

Text shorter than 50 characters after extraction/OCR is a hard failure. Text between 50 and 199 characters is accepted as sparse and logged as a warning. Encrypted PDFs are rejected. Image dimensions are bounded to reduce excessive memory use, and temporary OCR files are deleted after completion. These controls reduce malformed-input risk, although production deployment would additionally require malware scanning and stronger resource isolation.

QualityValidationService produces structured signals for questionable CV or JD content, including seniority/experience contradictions and salary/content inconsistencies. The design separates blocking errors from warnings so that the user

can correct poor-quality data without treating every imperfection as a processing failure.

CV ingestion implementation



CareerFit IT AutoPilot — thesis diagram

Figure 3.3. CV ingestion implementation pipeline

3.4 Text Normalization and TF-IDF Implementation

3.4.1 Text Normalization

TextNormalizationService removes HTML tags, replaces characters outside its English/Vietnamese alphanumeric pattern, lowercases text, splits on whitespace, removes tokens shorter than two characters, and filters a language-specific stop-word set. Language detection is a lightweight heuristic: text containing more than five Vietnamese diacritic characters is classified as Vietnamese; otherwise it is treated as English.

This implementation is deterministic and easy to inspect, but it is not a linguistic word segmenter. Vietnamese multi-syllable skills can be split into independent tokens, and punctuation-containing technology names such as C++, C#, .NET, or Node.js may lose distinguishing characters. These limitations are important when interpreting matching errors and motivate a future domain-aware tokenizer or controlled alias dictionary.

3.4.2 Static Corpus and Vector Construction

TfIdfService builds its IDF map once at application startup from a static seed corpus of representative IT terms. The corpus groups programming languages, frameworks, databases, cloud and DevOps technologies, architecture, testing, security, data/ML, collaboration, seniority, roles, and common Vietnamese IT words. A static corpus prevents scores from shifting whenever a user uploads one CV or adds one Job.

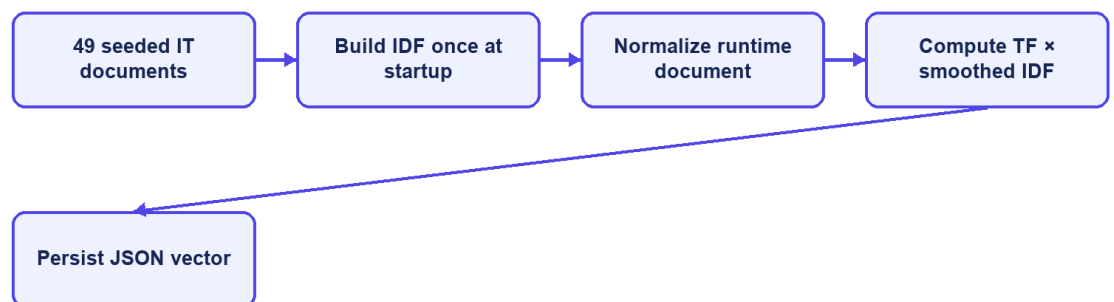
For a token list, term frequency is the token count divided by the total number of tokens. The implemented smoothed IDF is:

$$idf(t) = \log(1 + N / (1 + df(t))).$$

An unknown term receives $\log(1 + N)$, which treats it as rare and informative. The TF-IDF vector is persisted as JSON and reused during scoring. Cosine similarity iterates over the smaller vector for the dot product, computes both Euclidean magnitudes, and returns zero when either vector is empty.

The unknown-term policy has a practical trade-off. It preserves project-specific technology terms absent from the seed corpus, but a rare misspelling can receive the same high IDF treatment. Input validation, alias normalization, corpus versioning, and evaluation on realistic data are therefore necessary before treating the score as stable beyond the academic environment.

Static-corpus TF-IDF



CareerFit IT AutoPilot — thesis diagram

Figure 3.4. Seed-corpus initialization and TF-IDF construction

3.5 Matching and Recommendation Implementation

3.5.1 Scoring Service

ScoringService loads the CV vector from extractedTermsJson. For the Job, it prefers learnedProfileVectorJson when a non-empty learned vector exists; otherwise it uses the original tfidfVectorJson. Cosine similarity becomes the raw score, which is multiplied by 100 and rounded to two decimal places. The raw score is stored with six decimal places.

Table 3.3. Implemented score interpretation

Condition	Stored/displayed result
Score below 70	LOW label
Score from 70 to below 90	MEDIUM label
Score at least 90	HIGH label
Score from 35 to below 75 with at least three shared weighted terms	isPotential=true
Score from 35 to below 75 with compatible seniority and at least two shared terms	isPotential=true

Although configuration contains low, medium, and high boundary names, the current assignLabel method uses only the configured medium value (70) and high value (90); the configured low maximum (40) does not create a separate transition. POTENTIAL exists in the enum and schema, but current scoring returns LOW, MEDIUM, or HIGH and represents potential primarily through the separate boolean and reason. Chapter 4 must test these exact boundaries rather than infer behavior from configuration names.

Match reasons are the five highest-weight Job terms also present in the CV, with Job domain inserted first when available. The potential reason is a fixed explanation about transferable skills and career progression. These reasons are grounded in shared vector terms, but the potential explanation is heuristic rather than a learned causal explanation.

3.5.2 Matching Orchestration and Persistence

After CV vectorization commits, AfterCommitExecutor submits MatchingService.scoreAllJobsForCv by CV identifier to the bounded task executor. The service reloads the persisted CV, obtains eligible active Jobs, calls ScoringService for each pair, and creates or updates the unique Matching row. Reasons and potential metadata are serialized as JSON. Database uniqueness on (cv_id, job_id) prevents duplicate pairs, while conflict handling covers concurrent execution. The same boundary is used when a new or updated Job is scored against existing CVs.

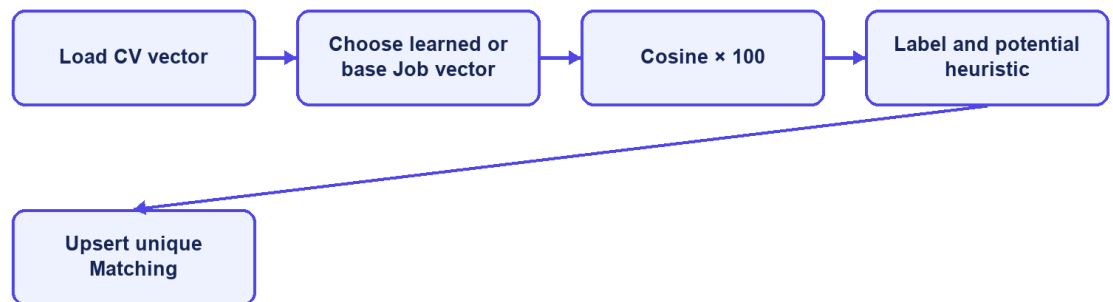
When a Recruiter creates or updates a Job, the service builds the Job vector and schedules post-commit scoring. MatchingQueryService shapes Candidate cards and

recruiter-oriented results, including pagination and metadata for empty, filtered, low-match, or tied-score states. CandidatePortfolioVisibilityService adds portfolio data only when the Candidate enabled showPortfolioAfterApply and has actually applied or moved beyond an invitation-only state; otherwise the response contains an explicit hidden reason. Stable score ordering remains important because rounded scores can be equal.

3.5.3 Recommendation Service

RecommendationService serves personalized Job results and similar-job retrieval. Candidate recommendations use profile/preferences and available matching information, while similar jobs can be retrieved publicly. The API and UI intentionally keep recommendations separate from the persisted application workflow. A recommendation can be viewed, skipped, or used to start an application without becoming an application automatically unless an enabled AutoFit policy separately authorizes that action.

Scoring and persistence



CareerFit IT AutoPilot — thesis diagram

Figure 3.5. Scoring and matching persistence flow

3.6 Feedback Learning with Rocchio

FeedbackService.submitFeedback resolves the Matching and authenticated Candidate, verifies that the Candidate owns the associated CV, and then upserts feedback by (matching_id, actor_id). Concurrent uniqueness conflicts are resolved by reloading and updating the existing row. Each event stores actor role, feedback type, and source channel and creates an audit entry. NOT_INTERESTED is stored but does not trigger Rocchio because it can represent preference rather than technical relevance. GOOD_MATCH, POTENTIAL, and BAD_MATCH trigger post-commit Job-vector learning.

RocchioService locks the Job for update and always loads the original Job TF-IDF vector as q . Positive examples are CV vectors attached to GOOD_MATCH or POTENTIAL feedback; negative examples come from BAD_MATCH. The service computes term-wise centroids and applies:

$$q_{new} = 1.0q + 0.75 \text{ positive_centroid} - 0.15 \text{ negative_centroid}.$$

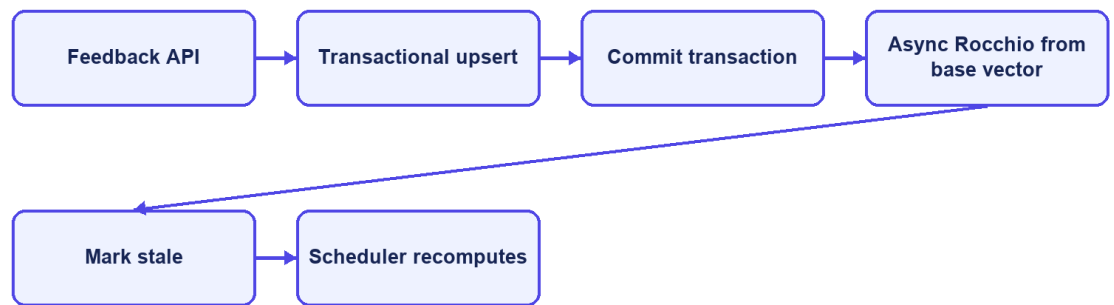
Negative weights are removed rather than persisted. The learned vector is stored in learnedProfileVectorJson. Every Matching associated with the Job is then marked needsRecompute=true. AutomationScheduler.recomputeStaleMatchings rescans these rows every 30 minutes and clears the marker only after successful scoring.

Using the original base vector and the complete feedback history makes recomputation repeatable for the same data and parameters. FeedbackService starts learning after the transaction commits, so Rocchio reads the saved feedback instead of racing the transaction that created it. Job locking prevents two updates from writing separate learned vectors at the same time, and integration tests cover this post-commit behavior.

Table 3.4. Feedback handling

Feedback	Persisted	Rocchio classification	Immediate learning trigger
GOOD_MATCH	Yes	Positive	Yes
POTENTIAL	Yes	Positive	Yes
BAD_MATCH	Yes	Negative	Yes
NOT_INTERESTED	Yes	Neither	No

Feedback implementation



CareerFit IT AutoPilot — thesis diagram

Figure 3.6. Feedback processing and post-commit learning

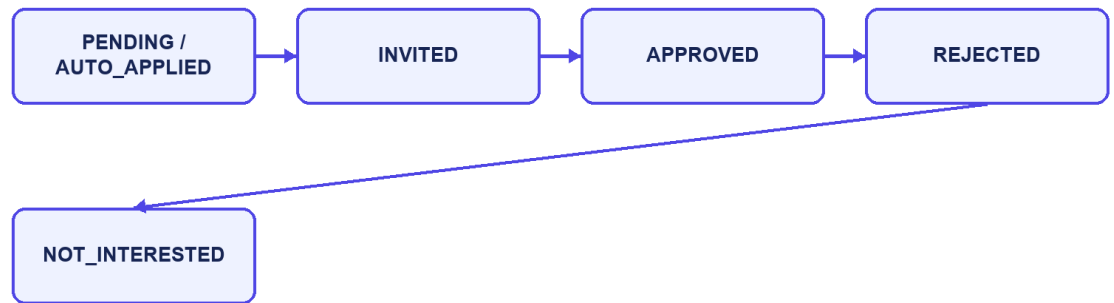
3.7 Application and Recruiter Workflow

`ApplicationService.submit` resolves the authenticated Candidate, requires an active Job, rejects an existing Candidate–Job application, and resolves either the requested CV or the Candidate's default CV. If the corresponding Matching exists, it is attached to preserve the score context at application time. `saveAndFlush` exposes uniqueness conflicts within the service transaction, which are converted into an HTTP conflict response. The service writes an audit event and requests Candidate and Recruiter notifications.

A Candidate lists owned applications through a paginated response and can withdraw an application unless it is already approved or rejected. Withdrawal changes status to `NOT_INTERESTED` rather than deleting history. A Recruiter must own the Job to list applicants or update status. Inviting a Candidate creates an `INVITED` application when no Candidate–Job record exists and returns the existing record when a concurrent or prior action already created it.

The response DTOs combine application state with selected Candidate, CV, and Matching fields without returning persistence entities directly. Portfolio fields are privacy-gated: a Recruiter receives them only after a qualifying application state and when the Candidate has enabled portfolio visibility after applying. List metadata communicates states such as no match, no filtered results, and ready, allowing the frontend to display a meaningful empty state.

Application state flow



CareerFit IT AutoPilot — thesis diagram

Figure 3.7. Application and invitation state transitions

3.8 AutoFit and Background Processing

3.8.1 Async Executor and Scheduler

AsyncConfig creates a bounded ThreadPoolTaskExecutor using configured core size, maximum size, queue capacity, and thread prefix. AfterCommitExecutor registers CV and Job matching work with Spring transaction synchronization and submits it only after commit. Mail sending and Rocchio learning can also leave the request thread, while persisted CV and Matching states allow clients to observe completion.

AutomationScheduler coordinates five tasks described in Chapter 2. Their delays, cron expressions, and time zone are read from app.scheduler properties. Per-item exception handling prevents one failed candidate or matching record from stopping an entire scan, while logs record the processing outcome.

3.8.2 Auto-Apply

AutoApplyService.runForPolicy resolves the policy owner, Candidate, and default CV and requires SCORING_DONE. It reads the top 20 Matches, considers active Jobs at or above the configured threshold, skips existing Candidate–Job applications, and creates at most three automatic applications per run. Database uniqueness is the final concurrency guard. Each created record is marked automatic, audited with the Matching and score, and followed by Candidate and Recruiter notification requests.

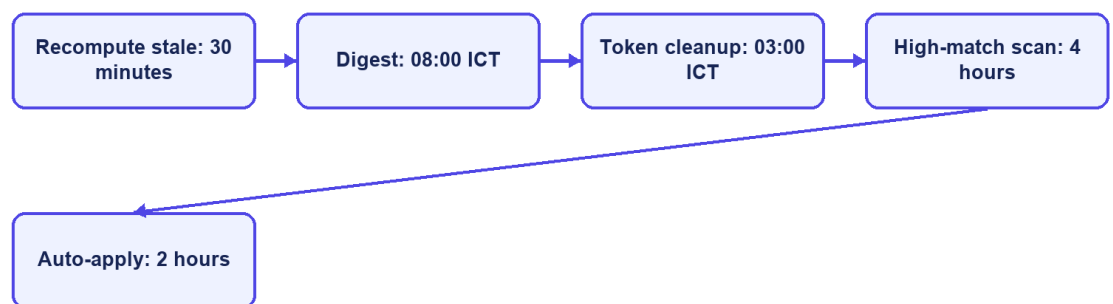
The current auto-apply implementation checks the supplied policy's threshold and enablement is handled by the caller selecting enabled policies. It does not

independently evaluate every notification control such as quiet hours or email quotas before creating an application. Those settings primarily affect notification policy. This distinction must remain explicit: notification gating and application authorization are related but not identical mechanisms.

3.8.3 Notification Guard and Delivery

NotificationPolicyGuard evaluates whether a message is allowed and writes delivery outcomes in independent transactions. It supports deduplication, timing, quota, cooldown, and skipped/failed reasons. After CV scoring, the backend can immediately send a high-match action email, a low-match notice, or a no-match notice; the scheduled scan remains a later safety path and deduplication prevents duplicate delivery. MailService sends asynchronously through SMTP when mail is enabled, while NoOpMailService supports local development without external delivery.

Scheduled automation



CareerFit IT AutoPilot — thesis diagram

Figure 3.8. Scheduler and AutoFit execution boundaries

3.9 Email Action and Audit Implementation

EmailActionService generates a 32-character token derived from UUID text for each supported action and stores it with recipient, optional Matching, type, status, and expiry. Match email actions include GOOD_MATCH, POTENTIAL, NOT_INTERESTED, and view links. Digest messages create actions for listed Matches and unsubscribe intent. Tokens remain valid for 72 hours.

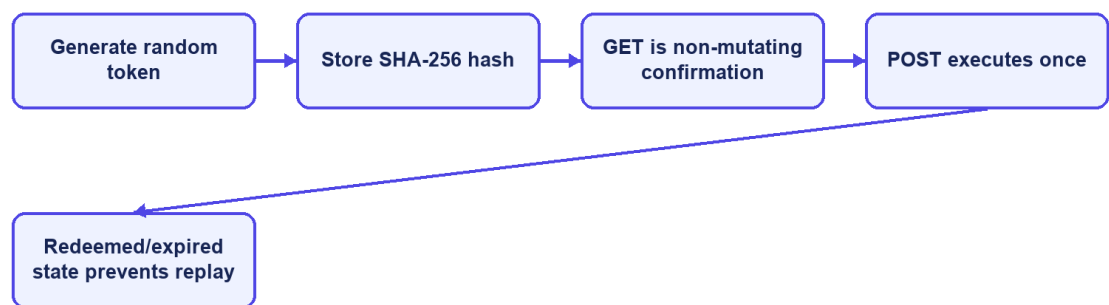
EmailActionController exposes a public confirmation and redemption flow because possession of the high-entropy token is the credential. The raw token is sent

to the user but only its SHA-256 hash is persisted. GET validates the token and renders a non-mutating confirmation page; POST performs the supported action, records redemption, and rejects unknown, expired, or already-used records.

This implementation removes the earlier state-changing GET and raw-token persistence findings. The remaining production concerns are rate limiting, deployment-specific link origin, secret rotation, mail-delivery monitoring, explicit handling of expired links, and protected audit review.

Audit records are written directly through `AuditLogRepository` in major services. An audit entry can contain actor type/ID, action, target, result, source channel, and JSON metadata. Examples include CV failure, application submission/withdrawal, candidate invitation, status update, feedback, and auto-apply. Direct repository calls are simple, but a centralized audit facade could standardize metadata, request IP/user-agent capture, redaction, and failure behavior.

Secure email-action implementation



CareerFit IT AutoPilot — thesis diagram

Figure 3.9. Hashed email-action token and confirm-then-POST flow

3.10 Persistence and API Implementation

JPA entities map the domain tables, while Spring Data repositories provide derived queries, pagination, locking queries, and explicit update/delete operations. Service methods use `@Transactional` for state changes and `readOnly=true` for queries where applicable. Flyway migrations V1–V15 create and harden the current schema, including hashed email-action tokens; Hibernate runs with `ddl-auto=validate`. This prevents application startup from silently inventing schema changes outside migration history.

Database constraints are treated as correctness boundaries rather than only documentation. Unique indexes protect email identity, default CV, Matching, Application, and imported Job hash. Check constraints restrict roles, labels, statuses, source channels, and salary modes. Version columns support optimistic concurrency on mutable core entities. JSONB stores vectors and variable metadata, while relational foreign keys preserve ownership and lifecycle relationships.

Controllers return shared API envelopes and DTOs instead of exposing entities. Validation annotations reject malformed requests near the API boundary, while service exceptions represent not found, forbidden, bad request, and conflict cases. OpenAPI is generated through springdoc. Public list endpoints and authenticated workspaces use pagination or bounded top-result queries to avoid unbounded payloads.

Table 3.5. Representative API-to-service mapping

API operation	Controller/service responsibility	Persistence effect
POST /api/cv/upload	Validate source, create CV, start ingestion	CV, file metadata, vectors, Matchings, audit
GET /api/matches/me/cards	Resolve Candidate/default CV and map ranked results	Read-only
POST /api/matches/{id}/feedback	Upsert feedback and trigger Rocchio	Feedback, audit, learned Job vector, stale flags
POST /api/applications	Validate ownership/state and create application	Application and audit
PATCH /api/automation/policy	Validate and update owned policy	Automation policy and version
POST /api/automation/auto-apply/run-now	Execute the same policy-based service on demand	Automatic applications, audit, notifications
GET /api/admin/audit-logs	Filter and paginate administrative audit view	Read-only

3.11 Frontend Integration

App.tsx defines public, Candidate, Recruiter, and Administrator routes. protectedRoute checks the loaded account role, while reload restoration calls /api/auth/me and clears an invalid session. Passwordless login has a dedicated magic-link verification route. Public routes include Jobs and employer details; Candidate routes cover asynchronous CV processing, CV management, recommendations, applications, analytics, automation, and settings; Recruiter and Administrator routes provide their role-specific workspaces. These checks improve navigation, but the backend remains the security boundary.

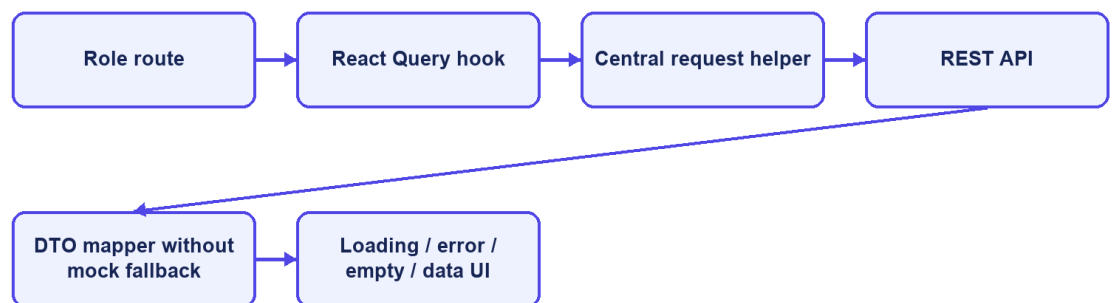
Frontend/src/lib/api.ts centralizes HTTP access. It reads VITE_API_BASE_URL with /api as the default, attaches the bearer token from sessionStorage, selects JSON headers except for multipart FormData, unwraps the

shared response envelope, and throws a normalized error for unsuccessful responses. DTO mapping functions convert backend shapes and labels into UI models. React Query hooks manage loading, refetching, caching, polling, and error state for server data.

The frontend stores the access token and account summary in sessionStorage, limiting persistence to the current browser tab. This still exposes the token to any successful cross-site scripting attack. The Content Security Policy reduces some exposure, but production hardening should evaluate short-lived tokens, refresh rotation, and HttpOnly secure-cookie alternatives according to the deployed architecture.

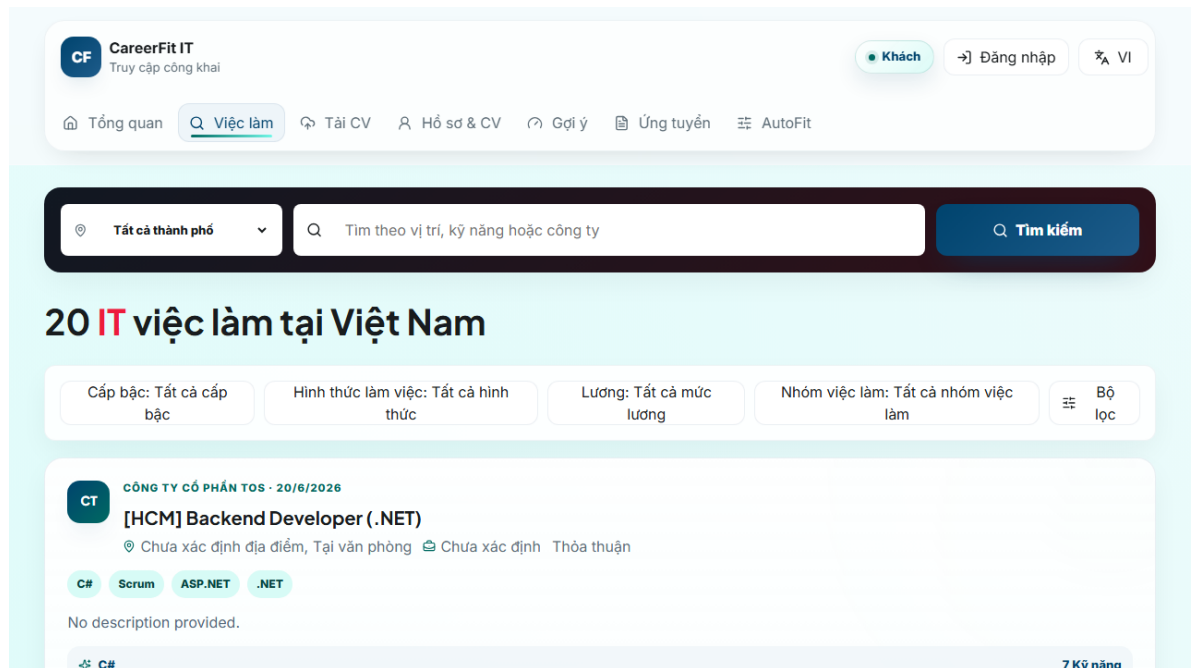
API-driven pages no longer substitute mock Job records when requests fail; they expose loading, retryable error, or empty states. The July 18 integration pass connected magic-link completion, session revalidation, CV status polling and management, the dedicated recommendation feed, and the market dashboard to backend contracts. Remaining UI gaps are limited to selected recruiter drill-down, employer self-service, automation pause/resume, telemetry, and administrative maintenance operations.

Frontend data flow

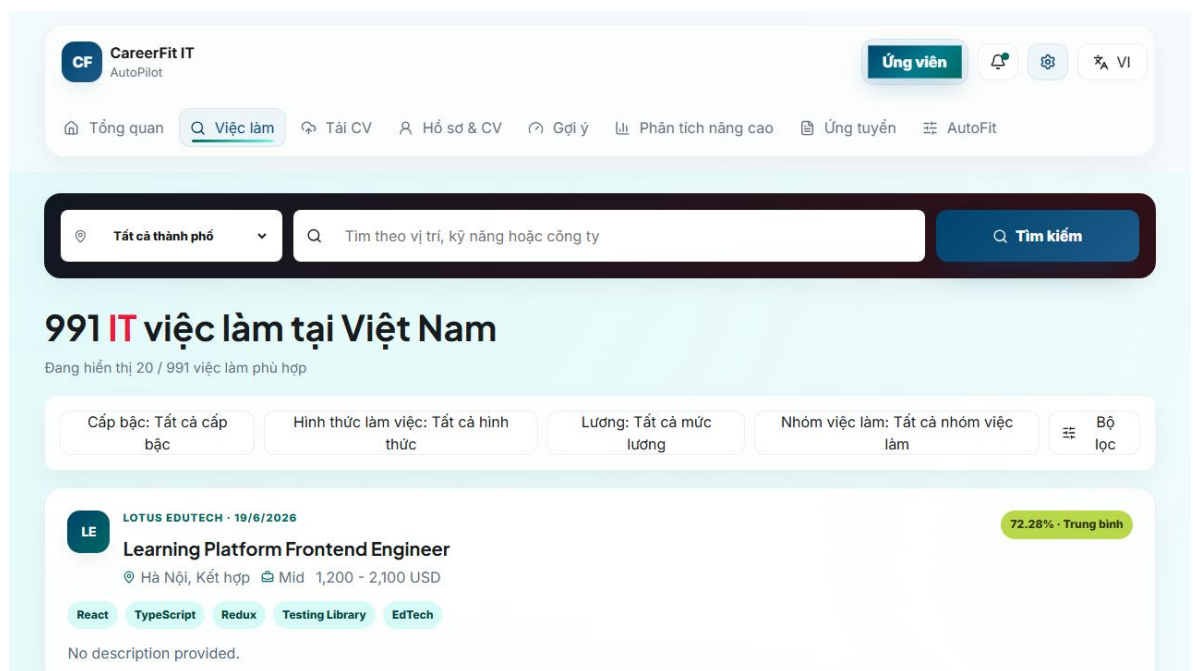


CareerFit IT AutoPilot — thesis diagram

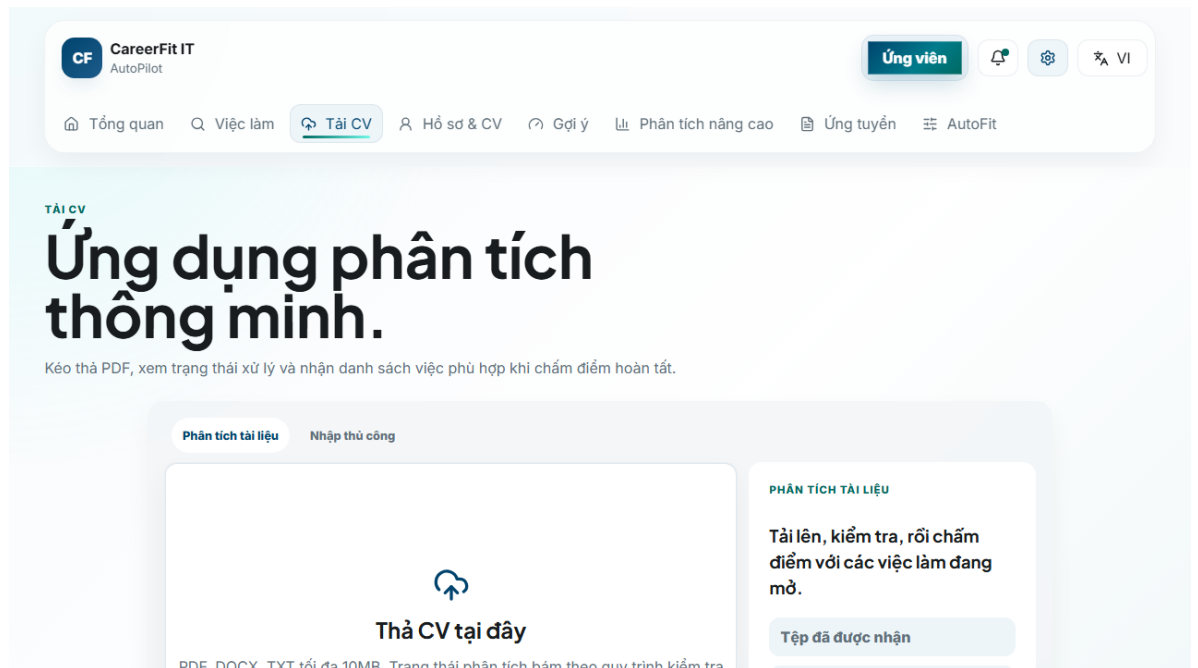
Figure 3.10. Frontend routes and API data flow



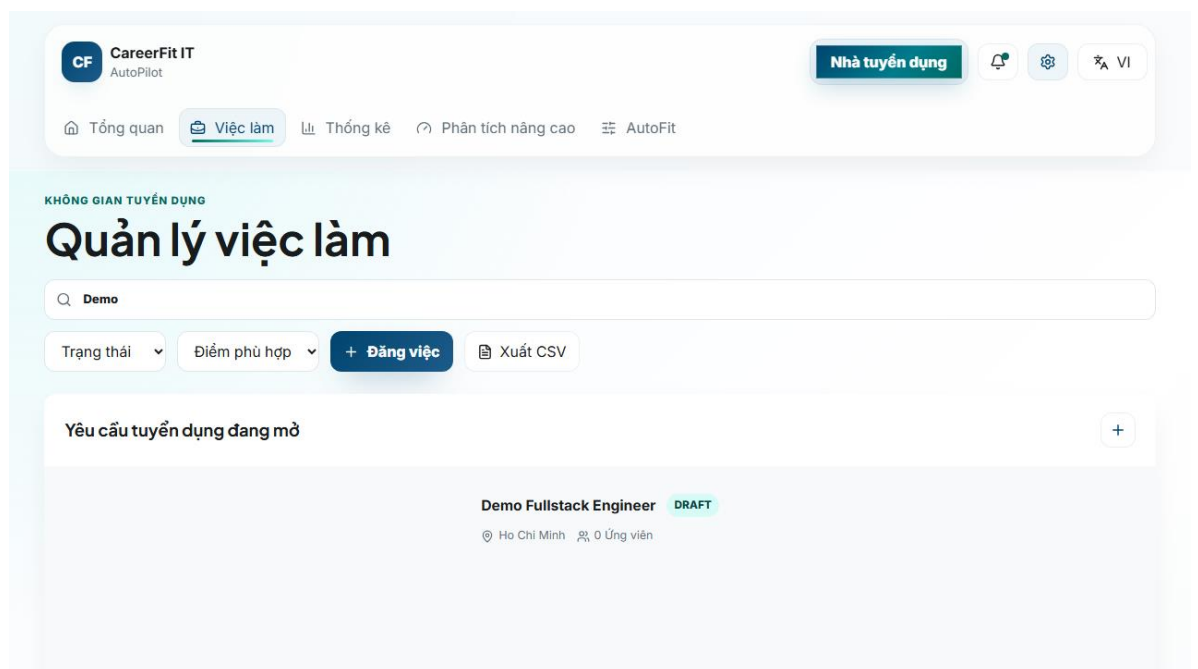
Screen 3.1. Public Job search



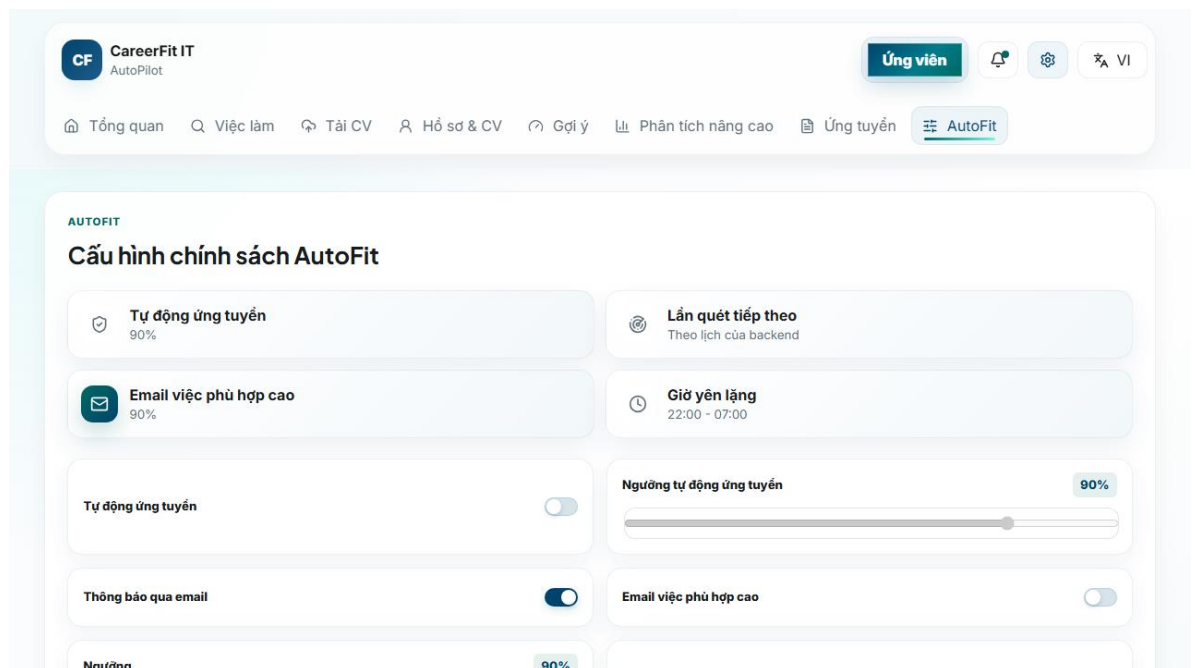
Screen 3.2. Candidate matching workspace



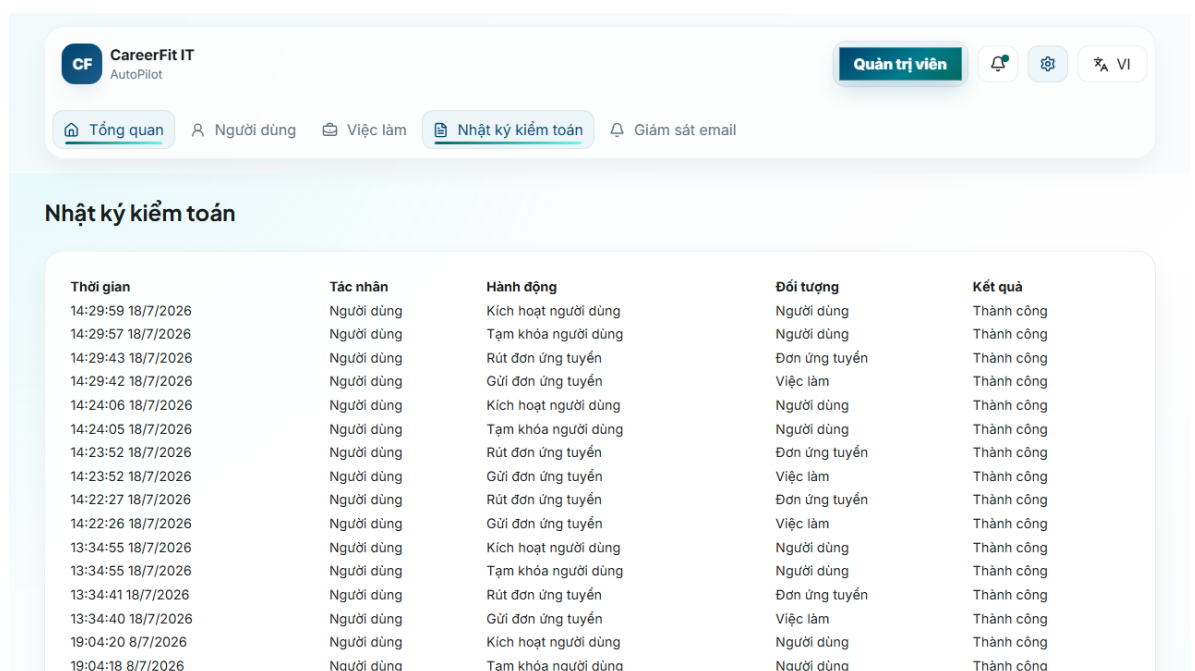
Screen 3.3. Candidate CV upload interface



Screen 3.4. Recruiter candidate workspace



Screen 3.5. Candidate AutoFit settings



Screen 3.6. Administrator audit view

3.12 Deployment and Observability Implementation

The default local runtime starts PostgreSQL through Docker Compose and can start the backend under the backend profile. Host execution connects to localhost:5433; container execution connects to postgres:5432. The backend image mounts /app/storage/cv, and environment variables override database, JWT, CORS, application URL, mail, OCR, and storage configuration. The frontend development server runs on 127.0.0.1:5173 and uses /api or a configured API base URL.

Actuator exposes health and Prometheus endpoints. Micrometer records HTTP request metrics with the application name tag and latency histograms. Console logs include the timestamp, thread, level, request ID when available, logger, and message. These features provide basic monitoring data, but an available endpoint does not by itself prove that the monitoring system works. Chapter 4 must separately check reachability, access rules, Prometheus collection, and dashboard or alert evidence.

Maven builds the backend and can execute JUnit, Spring Security, and Testcontainers tests. The frontend build runs TypeScript checking before Vite bundling. Playwright configurations support local and production-oriented E2E runs. Build and test commands, versions, environment, failures, and generated artifacts must be recorded when Chapter 4 results are finalized.

3.13 Implementation Limitations

Table 3.6. Verified implementation limitations

Area	Current limitation	Consequence or required improvement
Token action	Hashed token storage with GET confirmation and POST execution	Add rate limiting, origin controls, secret rotation, and delivery monitoring
Text processing	Whitespace tokenization and punctuation removal	Technology aliases and Vietnamese phrases may be lost
TF-IDF corpus	Static hand-curated seed corpus; unseen terms get maximum unknown IDF	Version corpus and test domain drift/misspellings
Labels	Configured low maximum is not used as a transition; potential is primarily a boolean	Align configuration names, schema label semantics, UI mapping, and boundary tests
Scheduler	All scheduler expressions use app.scheduler properties	Add schedule-boundary and time-zone integration tests
Async consistency	Learning is registered after transaction commit	Add production retry, conflict, and failure-recovery policy
Frontend session	JWT and account summary use sessionStorage	Evaluate HttpOnly cookie or refresh-token architecture and strengthen XSS controls
Frontend mapping	API-driven views no longer substitute mock Job fields	Maintain contract tests to prevent fallback data from returning

Area	Current limitation	Consequence or required improvement
File storage	Local path or Docker volume	Add malware scanning, encryption, backup, retention, and object storage for production
Audit	Services write audit rows directly	Centralize redaction, request context, schema conventions, and integrity policy

These limitations are included because the purpose of an implementation chapter is to explain the system that exists, not an idealized version. They also provide concrete hypotheses and negative cases for Chapter 4.

3.14 Chapter Summary

This chapter explained how the design is implemented in Spring Boot, React, PostgreSQL, and Flyway. It covered security, post-commit CV and Job processing, TF-IDF scoring, Candidate feedback with Rocchio, privacy-gated portfolios, application workflows, AutoFit notifications, hardened email actions, frontend integration, and runtime monitoring. Chapter 4 evaluates these parts using refreshed test and runtime evidence.

CHAPTER 4. TESTING AND EVALUATION

4.1 Evaluation Objectives

The evaluation was designed to answer four questions. First, does lexical scoring with Rocchio produce repeatable results and move holdout items in the expected direction after feedback? Second, do the backend modules and database migrations pass the unit, integration, security, and contract tests? Third, can the main Guest, Candidate, Recruiter, and Administrator workflows run through the integrated browser application? Fourth, what can be measured about local API latency, authorization, health, and monitoring without presenting the results as production validation?

The evaluation treats algorithm results, software correctness, browser workflows, security checks, and system health as separate areas. Passing one area does not mean that every other area also passes. In particular, the controlled Rocchio benchmark does not measure hiring quality with real production recruitment data, and a passing backend test suite does not prove that browser, monitoring, or deployment work is complete.

Backend, algorithm, frontend build, and Chromium test results were refreshed on July 18, 2026 (ICT). The observed Git HEAD was 65318fb0e0978574c9a04d9e54aecca5ba1eb241, but the worktree contained additional modifications and untracked files. Therefore, the results describe the evaluated working tree and are not reproducible from the commit hash alone unless the complete working-tree state is preserved.

4.2 Evaluation Environment

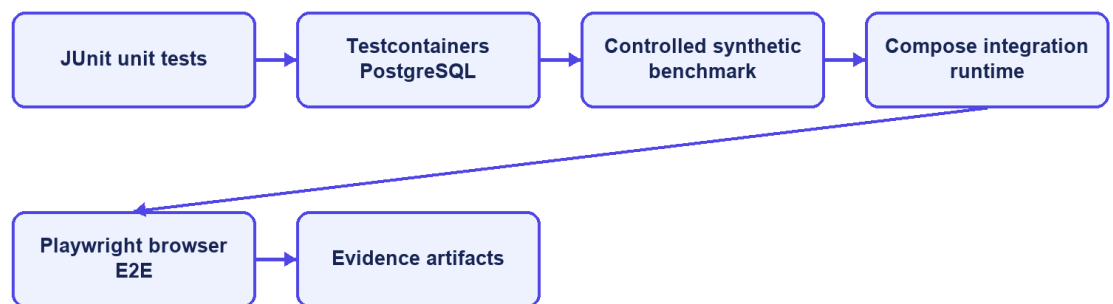
Table 4.1. Evaluation environment

Component	Observed version or configuration
Host operating environment	Windows, PowerShell
Java	21.0.1
Backend framework	Spring Boot 3.2.5
Maven	Repository Maven Wrapper
Node.js / npm	20.16.0 / 10.8.1
Frontend build	Vite 6.4.3, React 18.3, TypeScript 5.9
Docker / Compose	Docker Desktop server 29.5.3 / Compose v5.1.4
Test database	PostgreSQL 16.14 through Testcontainers or Compose
Testcontainers	1.21.4
Browser E2E	Playwright 1.61, Chromium project
Local backend	Host process on port 8080 connected to Compose PostgreSQL at localhost:5433
Local frontend	Vite development server on 127.0.0.1:5173

Docker Desktop was initially unavailable and was started before database-dependent verification. Testcontainers created isolated PostgreSQL containers for backend integration tests. Browser E2E used the existing local CareerFit database, which contained imported and seeded data. The public Job API reported 991 Jobs during the runtime check. This database is not the same dataset as the controlled algorithm benchmark.

The original command catalog is recorded in evidence/CHAPTER5_EVIDENCE_20260703.md. For the July 18 refresh, current Surefire XML reports, Playwright output, the production-build output, backend runtime logs, and evaluation/result.json provide the underlying artifacts.

Evaluation environments



CareerFit IT AutoPilot — thesis diagram

Figure 4.1. Evaluation environments and evidence sources

4.3 Dataset and Evaluation Design

4.3.1 Controlled Algorithm Dataset

AlgorithmEvaluatorTest loads evaluation/controlled-dataset.json. The dataset contains 50 Jobs and 100 unique CVs across IT domains. It defines 300 training pairs and 300 holdout pairs. The file hash observed during every repeated run was 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcf89fcc5cc2f2.

The benchmark is intentionally synthetic. Each Job is associated with a training-positive CV and a separate holdout-positive CV sharing a latent skill that is not emphasized in the original JD. The baseline ranks candidates using the original lexical representation. A positive feedback event supplies evidence from the training CV; Rocchio updates the Job vector; and the system reranks candidates. Metrics are